

Cultura DevOps: Integração e Entrega Contínuas

Guia Avançado: Fundamentos, Pipelines, Automação
e Boas Práticas Estritas



INSTITUIÇÃO EXECUTORA



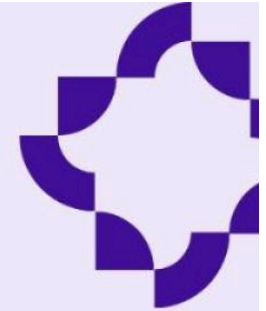
COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO





Sumário

- O que é CI/CD?
- Etapas Fundamentais de uma Pipeline
- Anatomia de uma Pipeline Real (GitHub Actions)
- Estratégias Avançadas de Deploy
- Boas Práticas Imperativas na Cultura DevOps



INSTITUIÇÃO EXECUTORA



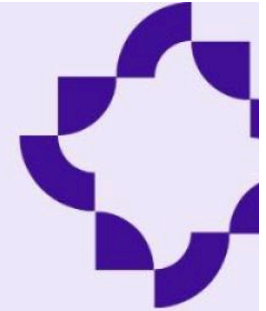
COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO





Cultura DevOps: Integração e Entrega Contínuas

Nesta apresentação, exploraremos a **Cultura DevOps** sob a ótica da **Integração Contínua (CI)** e **Entrega Contínua (CD)**. Abordaremos os fundamentos essenciais, as estratégias para a construção de *pipelines* robustos, a importância da automação e as melhores práticas estritas que garantem a eficiência e a confiabilidade no ciclo de desenvolvimento de software.



INSTITUIÇÃO EXECUTORA

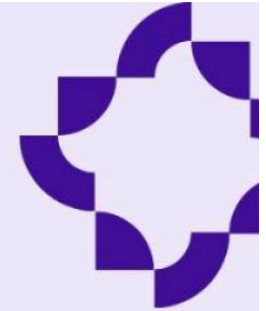


COORDENADORA



APOIO





Conceito Central: O que é CI/CD?

A esteira de **CI/CD** (Integração Contínua e Entrega Contínua) é a espinha dorsal da cultura DevOps. Ela automatiza os passos manuais que conectam o código à produção, mitigando erros humanos e gargalos. Os principais benefícios incluem: **redução de erros, eliminação de gargalos, aceleração da entrega de software e melhoria da qualidade do código.**



INSTITUIÇÃO EXECUTORA

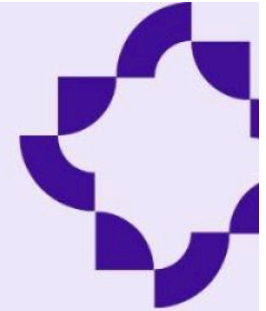


COORDENADORA



APOIO





Integração Contínua (CI)

A Integração Contínua (CI) é a prática essencial de integrar as alterações de código ao repositório central várias vezes ao dia. Cada *commit* realizado dispara automaticamente *builds* e testes, visando a detecção precoce de falhas de integração e garantindo que o código base permaneça sempre em um estado funcional e estável.



INSTITUIÇÃO EXECUTORA

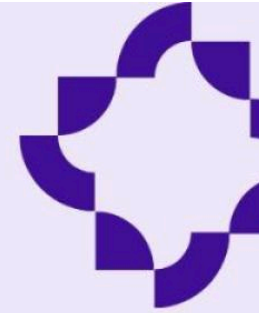


COORDENADORA



APOIO





Entrega Contínua (CD)

- A **automação** do *build*, teste e empacotamento garante que o código está sempre pronto para ser implantado.
- O **artefato** resultante está sempre em um estado implantável, aguardando a decisão de envio para produção.
- A **aprovação manual** é um requisito essencial para liberar o código ao cliente final, mantendo controle sobre o *deployment*.



INSTITUIÇÃO EXECUTORA

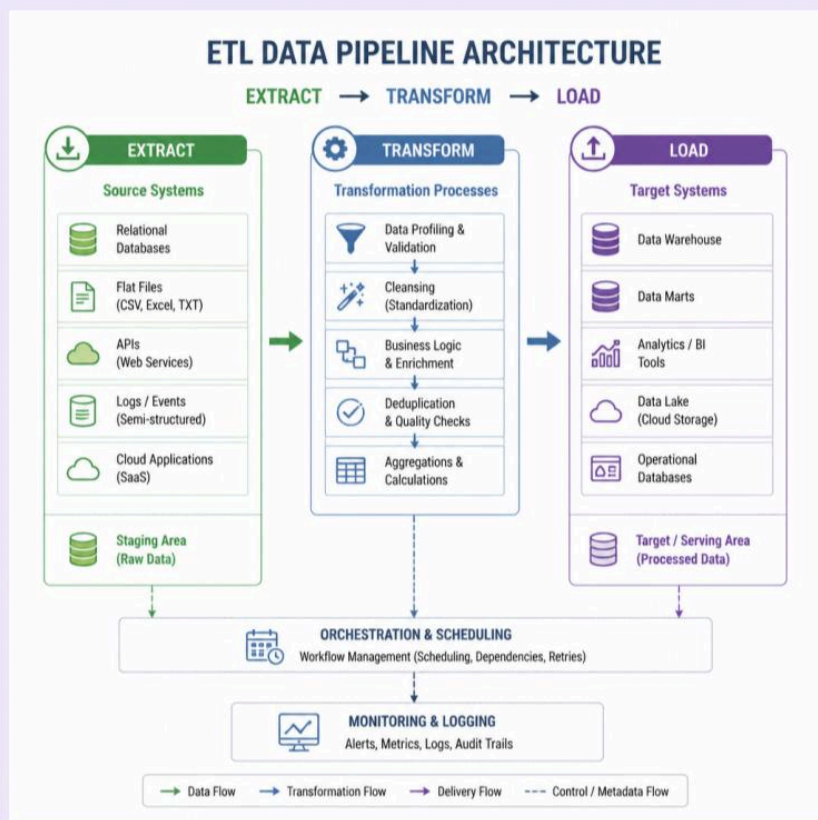


COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO



Arquitetura de pipeline ETL: a base da automação de dados.

Implantação Contínua (CD)

- A **Automação Completa** permite que toda alteração validada seja lançada diretamente em produção sem intervenção.
- A **Ausência de Intervenção Manual** é uma característica fundamental, eliminando aprovações manuais para a liberação.
- Exige **Alta Confiança** nos processos de teste e na robustez da automação para garantir a estabilidade do sistema.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO





Etapas da Pipeline CI/CD

Este slide detalha as fases fundamentais de uma pipeline CI/CD, descrevendo as ações executadas e seus objetivos práticos em cada etapa.

Fase	Ações Executadas	Objetivo Prático
1. Build	Compilação do código, download de dependências e empacotamento.	Garantir que o código não possui erros de sintaxe ou compilação.
2. Lint & Quality	Execução de Linters (ex: SonarQube, ESLint) e checagem de estilo.	Manter o padrão arquitetural do código e evitar débitos técnicos.
3. Test	Execução de testes unitários, de integração e checagem de cobertura.	Validar regras de negócio e garantir ausência de regressões.
4. Security (SAST)	Análise estática em busca de vulnerabilidades ou chaves vazadas.	Bloquear brechas de segurança antes da distribuição.
5. Package / Push	Geração da Imagem Docker e envio para o Registry.	Criar o artefato imutável oficial que será instalado nos servidores.
6. Deploy	Atualização do ambiente de destino (Staging ou Produção).	Disponibilizar a alteração de forma transparente para o usuário.

INSTITUIÇÃO EXECUTORA

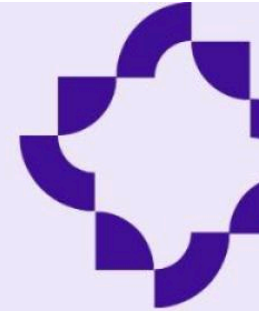


COORDENADORA



APOIO





Etapa Build

Esta fase foca na **compilação do código-fonte**, download de dependências e empacotamento. O principal objetivo é assegurar a ausência de erros de sintaxe ou compilação, garantindo que o software pode ser executado ou implantado.

Lint & Quality

Aqui são executados **Linters** (ex: SonarQube, ESLint) para checagem de estilo e conformidade. O objetivo é manter o padrão arquitetural do código e evitar o acúmulo de *débitos técnicos* que impactam a manutenção e escalabilidade.



INSTITUIÇÃO EXECUTORA

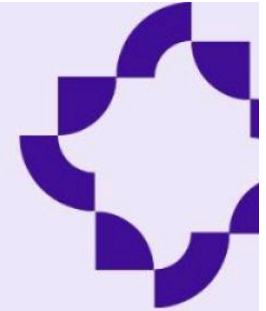


COORDENADORA



APOIO





Etapa 3: Teste

A fase de **Teste** na pipeline de DevOps foca na validação da funcionalidade do código. Isso envolve a execução de testes *unitários* (para módulos individuais), testes de *integração* (para interação entre componentes) e checagem da *cobertura* do código para assegurar que a maior parte do código-fonte seja exercitada. O objetivo principal é validar as regras de negócio implementadas e garantir a **ausência de regressões**.

Etapa 4: Segurança (SAST)

A fase de **Segurança**, especificamente via *SAST* (Static Application Security Testing), realiza a análise estática do código-fonte. Esta análise busca identificar vulnerabilidades de segurança potenciais (por exemplo, injeção de SQL, XSS) ou chaves vazadas. O objetivo é proativo: **bloquear brechas de segurança** e mitigar riscos antes que o código seja distribuído ou implementado em ambientes de produção.



INSTITUIÇÃO EXECUTORA

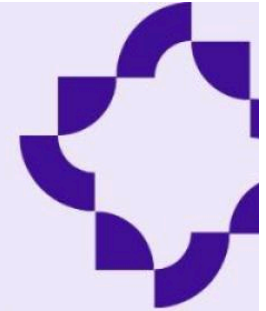


COORDENADORA



APOIO





Package / Push

Esta etapa foca na **geração de artefatos imutáveis**, como uma imagem Docker, e seu subsequente envio para um *Registry* (repositório de imagens). O objetivo central é criar e armazenar uma versão padronizada e estável da aplicação, que será a base para todas as futuras instalações e execuções em qualquer ambiente de destino. Isso garante a consistência do ambiente.

Deploy

O **Deploy** consiste na atualização do ambiente de destino, seja ele *Staging* (pré-produção) ou *Produção*, utilizando o artefato imutável gerado. Seu principal objetivo é disponibilizar as alterações da aplicação de forma **transparente** para os usuários finais, minimizando interrupções e assegurando a integridade e a continuidade do serviço. Técnicas como *Rolling Updates* ou *Blue/Green Deployment* são comuns aqui.



INSTITUIÇÃO EXECUTORA

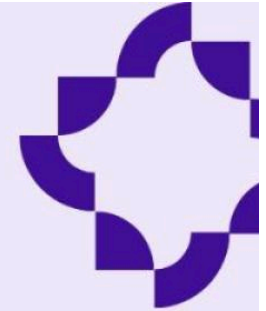


COORDENADORA



APOIO





Anatomia de uma Pipeline

- **GitHub Actions:** Gerencia pipelines nativamente no repositório através de arquivos declarativos YAML, salvos obrigatoriamente na pasta `.github/workflows/`.
- **Eventos (on):** Definem os gatilhos para a execução da pipeline, como um `push` de código ou a abertura de um `pull request`.
- **Jobs:** Agrupam os passos a serem executados em uma máquina virtual dedicada, como o `ubuntu-latest` ou `windows-latest`.
- **Steps:** São uma sequência de comandos shell ou *Actions* pré-definidas para construir, testar e publicar artefatos da aplicação.



INSTITUIÇÃO EXECUTORA

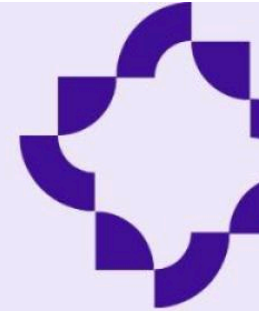


COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO



Exemplo Pipeline GitHub Actions

Este exemplo YAML ilustra uma pipeline de CI/CD, definida pelos gatilhos 'on' para branches específicas. Os 'jobs' orquestram a execução em ambientes como 'ubuntu-latest', enquanto os 'steps' detalham ações sequenciais. Inclui checkout de código, instalação de dependências, execução de testes e autenticação segura ('secrets') para Docker.



INSTITUIÇÃO EXECUTORA

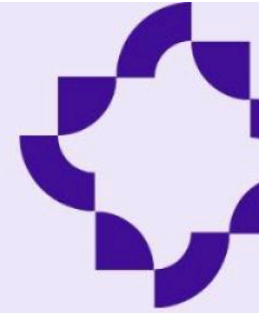


COORDENADORA



APOIO





Estratégias Avançadas de Deploy

- **Mitigação de Riscos:** Colocar código em produção exige técnicas para evitar indisponibilidades (*downtime*) e garantir a estabilidade do sistema.
- **Blue-Green Deployment:** Esta técnica consiste em manter dois ambientes idênticos em produção, alternando o tráfego entre eles para novas versões, o que facilita rollbacks rápidos.
- **Canary Deployment:** Implica lançar a nova versão para um pequeno subconjunto de usuários ou servidores, monitorando seu desempenho antes de escalar para toda a base.
- **Objetivos:** Ambas as estratégias visam reduzir o risco de falhas, minimizar o tempo de inatividade e permitir *rollbacks* eficientes em caso de problemas.



INSTITUIÇÃO EXECUTORA

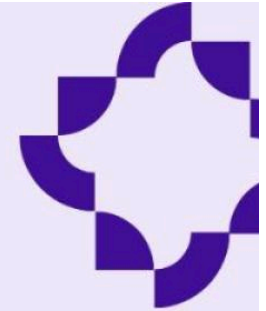


COORDENADORA



APOIO





Ambiente Blue

Este ambiente **ativo** mantém a versão atual e estável da aplicação. Ele continua a receber o tráfego real dos usuários, garantindo a disponibilidade do serviço enquanto a nova versão é preparada. O ambiente Blue também atua como uma rede de segurança, possibilitando um *rollback* **imediato** em caso de problemas com a nova versão.

Ambiente Green

Este ambiente **passivo** recebe a nova versão do código ou da aplicação. Testes finais, incluindo testes de regressão e performance, são executados neste ambiente isolado. Se a nova versão for validada com sucesso, o roteador ou *load balancer* direciona o tráfego dos usuários para o ambiente Green, tornando-o o novo ambiente **ativo**.



INSTITUIÇÃO EXECUTORA

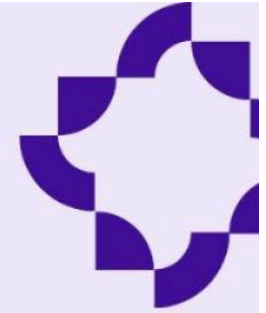


COORDENADORA



APOIO





Canary Deployment

- **Lançamento Gradual:** Uma nova versão é liberada inicialmente para uma pequena parcela do tráfego (ex: 5%), minimizando a exposição a potenciais problemas.
- **Monitoramento Contínuo:** A *pipeline* de CI/CD monitora ativamente o comportamento, desempenho e taxa de erros dessa amostragem em tempo real para avaliar a estabilidade.
- **Aumento Progressivo:** Se a versão se mantém estável, a porcentagem de tráfego é gradualmente aumentada (ex: 10%, 50%, 100%), permitindo observação contínua antes da adoção completa.
- **Mitigação de Riscos:** Este método permite testar em um ambiente de produção real com usuários finais, enquanto o risco de impacto adverso é drasticamente mitigado pela implantação controlada.



INSTITUIÇÃO EXECUTORA



COORDENADORA

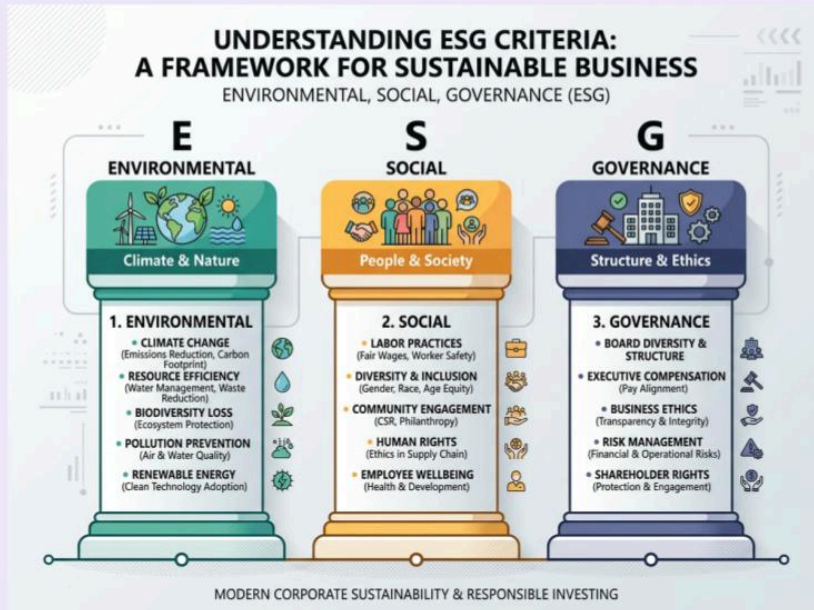


APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO



Regras Imperativas Corporativas



CrITÉRIOS ESG: sustentabilidade e governança para empresas responsáveis.

- **Imutabilidade de Artefatos:** Garanta que nenhum artefato (código, configuração, contêiner) seja modificado após ser construído, promovendo consistência e prevenindo desvios.
- **Segurança Absoluta (Secrets):** Trate credenciais, chaves e outras informações sensíveis como *secrets*, gerenciando-os em sistemas seguros e acessíveis apenas por entidades autorizadas.
- **Falhe Rápido (Fail Fast) e Cache:** Configure os pipelines para interromperem imediatamente ao detectar um problema, e utilize cache para acelerar reexecuções de etapas bem-sucedidas.

INSTITUIÇÃO EXECUTORA



COORDENADORA



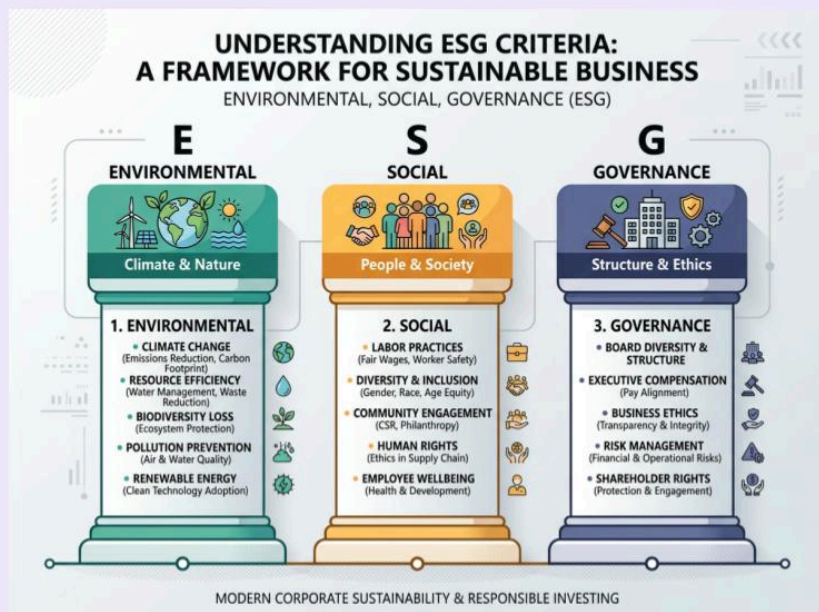
APOIO





Regras Imperativas Corporativas

- **Rastreabilidade Total:** Mantenha um registro completo e auditável de cada alteração, implantação e aprovação, essencial para depuração e conformidade regulatória.



Crítérios ESG: sustentabilidade e governança para empresas responsáveis.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO





Imutabilidade de Artefatos

A **imagem Docker** gerada no processo de *build* deve ser idêntica em todos os ambientes (desenvolvimento, homologação, produção). Não recompile artefatos entre ambientes; modifique apenas variáveis de configuração específicas para cada contexto.

Segurança de Dados

Nunca insira credenciais, senhas ou *tokens* diretamente em *scripts* YAML. Utilize **ferramentas de segredos** (e.g., GitHub Secrets, Vault) para injetá-los como variáveis de ambiente em tempo de execução.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO





Falhe Rápido

Implementar uma estratégia de **Falhe Rápido** (Fall Fast) em *pipeline* de CI/CD é crucial para otimizar o tempo de *feedback* e a eficiência do desenvolvimento. Isso envolve a ordenação de passos de validação, priorizando execuções rápidas, como verificações de *Lint* e *Testes Unitários*, antes de tarefas mais demoradas, como o *Build* completo do projeto. Além disso, a utilização estratégica de *Cache* para dependências externas é vital para acelerar repetidas execuções, garantindo que o *pipeline* mantenha sua velocidade e responda

Rastreabilidade Total

A **Rastreabilidade Total** garante a capacidade de identificar com precisão a origem de qualquer artefato implantado. Uma prática fundamental é taggear artefatos com o *hash* curto do *Git* (ex: `api:a1b23d`), evitando o uso de tags genéricas como `latest`. Essa abordagem permite que se identifique exatamente qual *commit* gerou um artefato específico em produção, facilitando a depuração, auditoria e reversão de versões, além de fortalecer a segurança e a conformidade do processo de *deployment*.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Dúvidas? Automatize com CI/CD!



Apresentamos as melhores práticas essenciais para pipelines CI/CD robustos e eficientes.

*Agora, abrimos o espaço para suas **perguntas** e **reflexões**.*

*Lembre-se: A **automação** é a chave para a eficiência e a inovação contínua na Cultura DevOps. Encorajamos você a aplicar os princípios de CI/CD, com \$git push origin master\$, em seus próprios projetos.*



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO

Conclusão

- CI/CD é a espinha dorsal do DevOps, automatizando o fluxo de código para produção e mitigando erros humanos.
- Pipelines CI/CD robustas possuem etapas como Build, Test, Security (SAST), Package e Deploy, garantindo qualidade contínua.
- GitHub Actions exemplifica pipelines reais, utilizando YAML para definir eventos, jobs e steps, incluindo a gestão de segredos.
- Estratégias como Blue-Green e Canary Deployment mitigam riscos de indisponibilidade, liberando software de forma segura e gradual.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO